

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
09.03.01 Информатика и вычислительная техника

направленность (профиль)
«Технологии разработки программного обеспечения»

Выпускная квалификационная работа

Разработка компьютерной игры стратегии с использованием ресурсов
искусственного интеллекта посредством Telegram-бота

Обучающегося 4 курса
очной формы обучения
Шульга Евгения Евгеньевича

Руководитель выпускной квалификационной
работы:
доктор педагогических наук, профессор,
заведующий кафедрой информационных
технологий и электронного обучения
Власова Елена Зотиковна

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ КОМПЬЮТЕРНОЙ ИГРЫ СТРАТЕГИИ В СРЕДЕ МЕССЕНДЖЕРА	5
1.1 Определение компьютерной игры как объекта программной разработки .	5
1.2 Мессенджер и искусственный интеллект в игровом процессе	7
1.3 Анализ существующих решений и систем – аналогов.	11
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА КОМПЬЮТЕРНОЙ ИГРЫ-СТРАТЕГИИ ПОСРЕДСТВОМ МЕССЕНДЖЕРА	15
2.1 Функциональные требования и архитектура программного продукта	15
2.2 Реализация игровой механики разрабатываемого продукта	24
2.4 Интеграция ресурсов искусственного интеллекта	31
2.5 Реализация пользовательского интерфейса	39
ГЛАВА 3. ТЕСТИРОВАНИЕ И ОЦЕНКА ЭФФЕКТИВНОСТИ РАЗРАБОТАННОЙ КОМПЬЮТЕРНОЙ ИГРЫ-СТРАТЕГИИ	46
3.1 Методика тестирования программного продукта.....	46
3.2 Проверка функциональности Telegram-бота и игровой логики.....	49
3.3 Оценка ресурсов искусственного интеллекта в игре и перспективы проекта	52
ЗАКЛЮЧЕНИЕ	55
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ	57

ВВЕДЕНИЕ

С развитием технологий искусственного интеллекта, расширением набора инструментов, предоставляемых популярными ресурсами под управлением роботизированного помощника, был создан большой пакет возможностей для реализации массы интерактивных программных продуктов на базе интернет – порталов, обеспечения для домашних компьютеров. Одним из наиболее массовых исполнений является разработка миниатюрных приложений на основе виртуальных сервисов в популярных мессенджерах, предоставляющих возможность взаимодействия пользователя в условиях конкретных игровых сред без установки отдельных программ на устройство.

Использование возможностей искусственного интеллекта может быть разным:

- мост между платформой по генерации медиа и текстовых файлов и мессенджером;
- виртуальный собеседник, точно имитирующий общение с человеком через доступные интерфейсы;
- куратор игрового процесса, способный задавать тон, а также сюжет при наличии определенных игровых условий.

Эти возможности предоставляют высокую вариативность при интеграции искусственного интеллекта в собственный продукт, лучшую адаптацию пользователя к продукту за счет общей вовлеченности.

Актуальность темы обусловлена сочетанием факторов тенденции к внедрению ресурсов искусственного интеллекта в самые непредсказуемые секторы и доступности современных мессенджеров. Особенно важным является изучение поведенческих черт различных языковых моделей искусственного интеллекта в рамках взаимодействия с человеком в среде игрового продукта, для анализа возможности построения сложной мыслительной логики, соответствия заданному шаблону поведения и форматам ответа.

Целью дипломной работы является разработка компьютерной игры в жанре стратегии с использованием ресурсов искусственного интеллекта на основе telegram – бота.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- Провести анализ основ разработки компьютерных игр со спецификой жанра стратегии и использованием ресурсов искусственного интеллекта;
- Проанализировать существующие системы – аналоги и сформировать на их основе выставляемые требования к системе;
- Спроектировать архитектуру компьютерной игры – стратегии в основе telegram – бота с использованием искусственного интеллекта;
- Разработать ключевые компоненты системы, а также их интеграции с ресурсами искусственного интеллекта для реализации игровых механик;
- Провести качественное тестирование итогового продукта с разбиением по видам тестирования и техникам тест дизайна, дать оценку реализации.

Объектом исследования является процесс разработки компьютерных игр с применением ресурсов искусственного интеллекта и возможностей современных мессенджеров.

Предмет исследования – разработанная компьютерная игра в жанре стратегии на основе telegram – бота с использованием искусственного интеллекта.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ КОМПЬЮТЕРНОЙ ИГРЫ СТРАТЕГИИ В СРЕДЕ МЕССЕНДЖЕРА

1.1 Определение компьютерной игры как объекта программной разработки

В настоящем разделе автор предоставляет определение компьютерной игры в жанре стратегия в контексте данной дипломной работы.

С технической точки зрения компьютерная игра представляет собой разновидность программного обеспечения, обеспечивающего для потребителя подготовленную интерактивную среду, ограниченную игровой условностью [1,2]. Под программным обеспечением, Согласно ГОСТ 19781-90, следует понимать совокупность документов, инструкций и алгоритмов, предназначенных для эксплуатации системы с целью достижения определенной задачи [1]. Основой каждой игры является грамотно построенная интерактивная симуляция, которая закладывается в игровые механизмы, направленные на решение поставленных задач и предоставление уникального эмпирического опыта [3].

С точки зрения информатики игра представляет собой классическое программное обеспечение, где пользователь вводит набор команд посредством доступных ему интерфейсов, а игровая среда формирует ответную реакцию на действие пользователя, построенную на основе определенных правил [4]. Итоговая разработка характеризуется эффективностью в удовлетворении пользователя при достижении поставленных целей согласно международному стандарту ISO 9241-11:2018 «Эргономика взаимодействия человек-система» [2]. Таким образом, каждый игровой опыт является по своему уникальным, а сама программа – динамический продукт, состояние которого непрерывно связано с действием игрока [5].

Поскольку мы говорим о построении игровой концепции, как о ключевом факторе успеха итогового продукта, немаловажным так же будет объявить приоритет в определении жанровой принадлежности игры, так как это напрямую

повлияет на общий пакет возможных механик из-за ограничений среды разработки [6]. В рамках данной выпускной квалификационной работы будет рассмотрена игра жанра стратегии с ориентиром на грамотное построение цепочки логических решений, управление персонажем в условиях ограниченных ресурсов и использование средств окружающей среды для достижения поставленных целей [7]. Для такого проекта одним из ключевых принципов будет являться масштабное планирование системы взаимодействия между отдельными игровыми сущностями, а также непосредственно с самим игроком [8]. Задача является нетривиальной, поскольку включает в себя выстраивание продуманной системы правил, завязанной на большом объеме взаимосвязанных параметров [9].

Если рассматривать итоговый продукт как разработку в среде мессенджера особо важным условием так же является адаптация игрового процесса под текстовую среду взаимодействия [10]. Поскольку продукт не сопровождается классическими для современных игровых реализаций интерфейсами для передачи информации непосредственному пользователю в виде звуковых и визуальных данных, проектирование необходимо рассмотреть с условием достаточности объема информации об игровом процессе в формате текстовых сообщений [11]. Аналогично должна быть построена логика взаимодействия с игрой для пользователя [12].

Таким образом разрабатываемый продукт представляет собой сложную программную систему объединяющую в себе технические и логические аспекты с наличием обширной интерактивной среды, устойчивой к условиям динамических изменений, согласованных общими принципами, обусловленными поставленной сценой [13]. Продукт включает в себя нетривиальную систему интерфейсов приема и передачи информации пользователю [14]. Принципы поставленные в основу задачи позволяют говорить о актуальности и значимости в контексте современных тенденций разработки [15].

1.2 Мессенджер и искусственный интеллект в игровом процессе

В настоящем разделе дипломной работы автор обосновывает выбор виртуального помощника – бота, как платформу для реализации игрового процесса. Так же рассматривается применение ресурсов искусственного интеллекта в области игрового процесса.

Современные платформенные решения предоставляют разработчикам большой спектр возможностей для реализации программных продуктов на основе их обеспечения [16]. Одной из таких платформ для разработчиков являются виртуальные помощники, а именно боты в мессенджерах, благодаря открытым интерфейсам, широкой кодовой базе и наличию инструкций для молодых разработчиков они являются одним из самых перспективных направлений разработки в наше время [17].

Под виртуальным помощником понимается специализированный набор алгоритмов, реализованный в функциональный программный продукт в среде мессенджера имеющий в качестве интерфейсов текстовый диалог с пользователем, наборы встроенных команд [18]. Сам по себе виртуальный помощник не представляет собой индивидуальную разработку, обеспечивающую полный функционал, он выступает мостом между пользователем и размещенным удаленно на выделенных серверах алгоритмом, обрабатывающим входящие запросы в соответствии с заложенной логикой [19]. Таким образом, виртуального помощника следует рассматривать как клиент в классической модели клиент – серверного взаимодействия [20].

Интерактивная составляющая виртуального помощника обеспечивается за счет открытого программного интерфейса, для данной дипломной работы – Telegram Bot API, предоставляющего разработчику набор правил по общению с сервисом и обработки всех возможных действиях пользователя в рамках текстового диалога. Выбранная система поддерживает через открытый интерфейс возможности по отправке медиа и аудио данных, специализированных интерактивных элементов, прием входящих запросов через

инструменты long polling и webhook [21]. В первом случае, сервер периодически обращается к текстовому диалогу с пользователем и проверяет наличие новых событий, а во втором сама платформа отправляет обновления на указанный в виртуальном помощнике адрес [22]. Для масштабного серверного решения рекомендуется использование webhook-механизма для оперативной обработки [22].

Одним из ключевых преимуществ для реализации логических и стратегических игр с пошаговым взаимодействием является удобный текстовый интерфейс, для которого удобно реализовывать поэтапное взаимодействия пользователя на модели перехода состояний [23]. Интерактивность обеспечивается через ввод текстовых сообщений и команд, которые затем интерпретируются в игровые решения, изменяя состояние окружающей пользователя среды [12].

Удобство пользования для непосредственного пользователя является отсутствие необходимости в установке отдельного программного обеспечения и регистрации на сторонних платформах [24]. При этом показатель активности пользователей в мессенджерах в медиане составляет от 2,5 до 4 часов в день и напрямую коррелирует от возрастной категории, где молодое поколение (выборка от 18 до 30 лет) тратит на общение и потребление информации через современные мессенджеры в среднем от 8 до 9 часов ежедневно, при этом процент цифровизации в России для людей старше 12 лет составляет ориентировочно 78 % [25]. Стремительно растущая аудитория сервисов моментального обмена сообщениями составляет более 96 миллионов пользователей ежемесячно [26]. Указанные показатели свидетельствуют о минимальном пороге входа для современного пользователя и общей осведомлённости о базовых механизмах работы мессенджеров, а следовательно, освоение интерфейса не требует значительного времени [24].

Немаловажным замечанием так же будет большая кроссплатформенность выбранной системы, платформа поддерживается на всех современных операционных системах и устройствах [21]. Следовательно процесс разработки

значительно упрощается и итоговые затраты на обеспечение единого пользовательского опыта будут минимальны [27]. Так же следует отметить удобство сопровождения программного продукта в будущем: классическая модель распространения программного обеспечения для устройств предполагает согласие на автоматическое обновление или регулярное ручное [28]. В случае выбранной платформы этот процесс является бесшовным для пользователя и происходит удаленно на серверной среде [22].

Несмотря на вышеперечисленные преимущества, платформа имеет под собой и ряд недостатков в сравнении с классическими компьютерными играми, а именно ограниченность в интерфейсах взаимодействия с пользователем, не предназначенность для реализации сложных анимированных и мультимедийных систем [29]. Возможности визуального восприятия игрового процесса ограничены аудио, медиа контентом и web-апп на базе существующих интернет – ресурсов [21]. Однако в представлении разработки стратегической пошаговой игры согласие с такими ограничениями отчасти являются рациональным подходом в разработке, обеспечивая должную концентрацию внимания пользователя на игровых механиках, при должном подходе не уменьшая процент вовлеченности современного потребителя к продукту [30]. Таким образом, простая техническая реализация, но объемный и проработанный игровой процесс сводятся к балансу.

Подводя итог анализа telegram – бота как платформы для реализации игрового процесса, можно сказать, что виртуальный помощник на ресурсах мессенджера – удобная и технически обоснованная платформа для реализации компьютерных игр с уклоном в пошаговое взаимодействие пользователя с игровой моделью [17]. Использование мессенджера как платформу обеспечивает высокий уровень доступности для пользователей по всему миру за счет процессов цифровизации и кроссплатформенности [24]. Отсутствие необходимости в установке, общая осведомленность и среднее время экрана в объеме 8-9 часов по медиане обеспечивает высокую перспективность такого направления в разработке [25].

Текущий объем предоставляемых услуг искусственным интеллектом и тенденции развития инфраструктуры для развертывания языковых моделей все чаще направляют бизнес в сторону интеграции ресурсов искусственного интеллекта с выстроенными процессами [31]. Использование технологий языковых моделей занимает немалое место в разработке современных программных систем [32]. Применение встречается на всех этапах жизненного цикла разработки программного изделия охватывая задачи, связанные с подготовлением системной и бизнесовой аналитики, реализацией продукта и проведением эталонного тестирования за счет применения агентов искусственного интеллекта [33] точно так же, как модели помогают в разработке программных систем, они учувствуют и в логике продукта [34]:

- автоматизация принятия решений;
- генерация информационного наполнения;
- оркестрация логических цепочек продукта.

Одним из наиболее распространенных направлений в сфере искусственного интеллекта является генерация текста [35]. Услуги по генерации текста для игровых систем на платформе мессенджера являются особенно важным аспектом, поскольку значительная часть игровой логики заключается в текстовом взаимодействии пользователя с виртуальным помощником [36]. Технология может быть применима для генерации игровой модели, подготовки заготовленных игровых персонажей, реакции на пользовательские действия [37].

В игровой разработке ресурсы искусственного интеллекта также применимы как механизм управления игровыми персонажами и интеллектуальной составляющей игрового процесса [38, 39].

Таким образом, использование ресурсов искусственного интеллекта в компьютерных играх представляет собой перспективное направление, позволяющее расширить функциональные возможности продукта, повысить вариативность игрового процесса и обеспечить более глубокое взаимодействие пользователя с игровой средой [40].

1.3 Анализ существующих решений и систем – аналогов.

В настоящем разделе дипломной работы автор проводит сравнительный анализ систем – аналогов в отношении разрабатываемого продукта и формирует список требований к продукту со стороны современного пользователя на основе анализа.

Изучив существующие конкурирующие продукты, можно вывести наиболее удачные практики, составить типизированный список недостатков, выделив основные требования к разрабатываемой системе [41]. Для данной дипломной работы будут рассмотрены существующие решения игрового характера на платформе telegram – бота с текстовым или упрощенным интерфейсом.

В экосистеме популярного мессенджера базируется значительное количество виртуальных помощников, реализующих различные игровые модели [17]. Среди большого спектра решений предлагается рассмотреть наиболее распространенные, в которых фигурируют знакомые множеству людей игровые конфигурации:

Программный продукт «WerewolfBot» реализует социально-логическую игру «Оборотень» для групповых диалогов [42]. Это психологический пошаговый ролевой продукт с детективным сюжетом. В рамках игрового процесса, игроки разделяются на несколько групп: люди и оборотни. Первые должны вычислить и устранить преступников, а вторые – избавиться от всех горожан. Игра состоит из чередующихся фаз, который регулирует ведущий, который не участвует в игровом процессе, лишь направляет его течение:

- Ночь: город «засыпает», оборотни договариваются о решении, кого они хотят исключить на текущей итерации, а затем делают решение активные роли горожан, которые могут повлиять на конечный результат действий оборотней.

- День: город «просыпается», происходит обсуждение произошедшего в предыдущей итерации, участники делятся подозрениями, строят теории.

Бот в данном случае выступает в качестве оркестратора и управляет распределением ролей игроков, голосованием, сменой игровых состояний «день – ночь». Продукт демонстрирует успешный пример реализации многопользовательского решения, однако механики не включают в себя использование ресурсов искусственного интеллекта.

Программный продукт «TruthOrLieBot» представляет собой программную реализацию популярной социальной игры «Правда или ложь» [43]. Правила игры заключаются в поочередной задаче вопроса, что выберет следующий игрок: рассказать правду или выполнить заданное действие. Если участник отвечает: «Правда», то он должен будет правдиво ответить на вопрос, который ему будет задан. Если он выберет действие, то должен будет выполнить действие, которое запросит предыдущий участник. После того как он проходит свою итерацию, ход переходит к следующему участнику. Бот выполняет роль оркестратора игрового процесса и не участвует в нем непосредственно, помимо прочего, продукт не использует технологии искусственного интеллекта, решение полностью построено на алгоритмической системе перехода состояний игры. Бот интересен в разрезе того, что за простой технической и логической составляющей, обладает высоким уровнем заинтересованности у пользователей мессенджера за счет построения игровой модели на основе знакомой многим социальной игры.

Программный продукт «PlayQuestBot» рассматривает текстовую пошаговую игровую модель с использованием искусственного интеллекта для реализации игрового процесса в формате приключения и подразумевает взаимодействие в формате текста, возможный набор которых определяется алгоритмами принятия решений искусственного интеллекта и ввод которых осуществляется путем выбора кнопок из встроенного меню [44]. Представленное решение интересно интеграцией ресурсов искусственного интеллекта в игровой процесс в качестве регулятора игрового окружения.

Помимо программных продуктов реализованных на платформе мессенджеров, целесообразно рассмотреть решения с использованием текстового или упрощенного пользовательского интерфейса на базе других платформ или операционных систем [45]. Такие решения представляют интерес с точки зрения реализации игровых механик и возможности адаптации игровых условий в рамках разрабатываемого продукта.

Программный продукт «Reigns» представляет собой продукт разработки мобильных приложений, в которой игрок управляет королевством, определяя решения в различных многосторонних вопросах, игрок влияет на четыре ключевых параметра: церковь, народ, армия и казна [46]. Игровой продукт интересен лаконичным визуальным интерфейсом, при его минималистичной направленности, продукт успешно передает весь необходимый пакет информации пользователю для принятия решений. Механика выбора из нескольких заранее определенных вариантов хорошо адаптируется к формату текстового диалога в мессенджере, однако в оригинальной версии игры наибольший пользовательский интерфейс достигает за счет графического интерфейса, реализация которого будет ограниченной на выбранной платформе.

Программный продукт «Dark Room» — это браузерный текстовый продукт, сочетающий элементы стратегии и управления игровым персонажем в условиях ограниченных ресурсов [47]. Пользователь управляет поселением, распределяет ресурсы и принимает стратегические решения для развития колонии. Игровая модель демонстрирует возможность реализации стратегического жанра без наличия визуальной составляющей, при наличии высокого пользовательского интереса, однако не включает в себя использование ресурсов искусственного интеллекта.

На основе проведенного обзора можно выделить ключевые характеристики существующих решений в виде сравнительного анализа. В качестве критериев сравнения выбраны следующие параметры: стратегические механики, использование искусственного интеллекта, формат интерфейса, адаптивность к виртуальному помощнику, пользовательский интерес.

Таблица 1.1 – Сравнительный анализ игровых систем

Характеристика	WerewolfBot	TruthOrLieBot	PlayQuestBot	Reigns	Dark Room
Стратегические механики	да	нет	нет	да	Да
Использование искусственного интеллекта	нет	нет	да	нет	Нет
Формат интерфейса	Текст	Текст	Текст	Визуал	Текст
Адаптивность для мессенджера	Да	Да	Да	Частично	Да
Пользовательский интерес	Большой	Большой	Средний	Большой	Средний

Сравнительный анализ позволяет сделать несколько выводов:

- Использование ресурсов искусственного интеллекта в области компьютерных игр на основе платформы мессенджера непопулярно.
- Сочетание стратегических механик и ресурсов искусственного интеллекта в игровой процесс не рассмотрено продуктами.

Подводя итог по первой главе, следует сказать, автор настоящей выпускной квалификационной работы разрабатывает уникальный продукт в виде компьютерной игры стратегии с использованием ресурсов искусственного интеллекта, о чем говорит проведенный сравнительный анализ, так как в его рамках не было обнаружено подобных систем. Выбранная платформа, игровая концепция и ресурсы искусственного интеллекта являются обоснованными в вышестоящих главах решениями, призванными восполнить недостающую на текущий момент нишу программных продуктов.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА КОМПЬЮТЕРНОЙ ИГРЫ-СТРАТЕГИИ ПОСРЕДСТВОМ МЕССЕНДЖЕРА

2.1 Функциональные требования и архитектура программного продукта

В настоящем разделе дипломной работы автор формирует список требований, которым отвечает разработанный программный продукт, а также описывает выстроенную архитектуру.

На основе проведенного обзора существующих смежных решений и сравнительного анализа, будет рассмотрена программная реализация социально-психологической игры «Мафия» с элементами стратегического взаимодействия между участниками партии реализуемая на платформе популярного на текущий момент мессенджера Telegram с использованием ресурсов искусственного интеллекта.

Выбор обусловлен рядом причин, прежде всего, игровая концепция хорошо адаптируется под доступные интерфейсы взаимодействия пользователя с продуктом – текстовый диалог с виртуальным помощником. Гипотеза была успешно подтверждена уже существующим программным продуктом «WerewolfBot», ознакомление с которым было проведено в рамках обзора. Игровой процесс строится так же на последовательной смене состояний, прорабатывании сложных логических цепочек стратегических действий, активном взаимодействии между участниками. В отличие от рассмотренных выше игровых концепций, «Мафия» без упрощений размещается в среду группового диалога: при добавлении интерактивных элементов в виде модульного меню и встроенных кнопок с сохранением единообразного и стилистически правильного интерфейса сохраняется высокий уровень заинтересованности пользователей за счет знакомых элементов интерфейса.

Ключевым аргументом, который повлиял на итоговый выбор, является выраженная стратегическая составляющая игры: принятие решений на основе

анализа поведения участников и попытка предугадать дальнейшие действия противоположной команды с высокой степенью влияния на игровой процесс.

Использование ресурсов искусственного интеллекта в данном случае дополнительно отразит поведенческие черты различных языковых моделей в условиях строго выстроенной концепции и строго прописанных правил, это позволит рассмотреть вопрос о принятии решения алгоритмами искусственного интеллекта при возникновении стрессовых ситуаций и заглянуть за пределы машинного кода далеко за весовую таблицу моделей. С помощью искусственного интеллекта планируется реализовать генерацию игровых персонажей, игровых условностей и текстового сопровождения игры, а также управления сгенерированными игровыми персонажами помимо непосредственного пользователя.

Таким образом, выбор игры «Мафия» для реализации является наиболее обоснованным с точки зрения платформы, так как обладает самыми высокими показателями в отношении адаптивности в проведении игрового процесса за основой текстового интерфейса. Аналогично, с точки зрения жанровой составляющей выбор является оптимальным несмотря на то, что направление у игровой концепции социально – психологическое, элементы стратегического взаимодействия в условиях ограниченных ресурсов являются ключевой составляющей «Мафии».

Предполагается, что пользователю не потребуется обладать специализированными знаниями для запуска и ведения игровой сессии, так как общение с виртуальным помощником должно быть интуитивно понятным и осуществляться через стандартные средства Telegram – команды, кнопки, веб – приложения.

Для непрерывного игрового процесса и корректной работы всего приложения в целом хранится следующий набор данных:

Данные пользователя приложения:

- уникальный цифровой идентификатор пользователя Telegram;
- отображаемое имя профиля;

Данные игровой сессии:

- идентификатор партии;
- активное состояние;
- номер итерации;
- конфигурация игровых ролей;
- игровой режим;
- количество игроков;

Данные участников игрового процесса:

- Идентификатор участника;
- Отображаемое имя;
- Флаг искусственного интеллекта;
- Роль;
- Паспорт персонажа;
- Игровой статус;
- Память;

Подход к реализации системы хранения данных и управления данными обеспечивает целостность активных игровых партий, долгосрочное хранение результатов завершённых партий, настроенных пользовательских профилей и игровых персонажей.

На основе проведённого анализа предполагаемого пути использования разработанного программного продукта целесообразно обозначить строгие сформированные функциональные требования. Система обеспечивает:

- Автоматическую регистрацию новых пользователей продукта;
- Сохранение данных пользователя продукта;
- Отображение главного меню;
- Отображение раздела справочной информации и поддержки;
- Конфигурирование новой игровой сессии;
- Запуск игровой сессии;
- Автоматическую генерацию уникальных игровых условий;
- Автоматическую генерацию игровых персонажей;

- Автоматическое распределение ролей для участников;
- Предоставление актуальной информации о состоянии партии;
- Бесшовный переход состояний партии;
- Корректную обработку сценариев исключения участника;
- Синхронную обработку команд пользователя;
- Проведение голосования;
- Обработку результатов перехода активных фаз;
- Сохранение активного состояния сессии;
- Завершение игровой сессии при достижении условий победы;
- Вывод итогов сессии;
- Долгосрочное хранение результатов сессии;
- Формирование запросов на генерацию игровых условностей;
- Формирование запросов на генерацию игровых персонажей;
- Фильтрацию и выборку ответов искусственного интеллекта;
- Корректную обработку сценариев недоступности модуля искусственного интеллекта.

Рассмотрим итоговую архитектуру разработанного продукта в виде компьютерной игры – стратегии на платформе мессенджера с использованием ресурсов искусственного интеллекта.

Для серверной части продукта был выбран язык программирования Python версии 3.11+. Выбор обоснован масштабной кодовой базой, наличием популярных библиотек, расширяющих базовые возможности языка программирования и необходимых для построения игрового процесса. Python удобен в вопросах описания больших ролевых моделей на основе ООП программирования, проведения автоматизированного тестирования и клиентов для общения с ресурсами искусственного интеллекта.

Список серверных библиотек с описанием их назначения для проекта:

- Aiogram 3.13, взаимодействие с открытыми интерфейсами мессенджера с поддержкой асинхронного взаимодействия, используется для разработки telegram – ботов;

- Aiohttp 3.9, поддержка мини-приложений мессенджер для построения пользовательского интерфейса через технологию «Webapp» с поддержкой WebSocket соединений;
- SQLAlchemy 2 + Asyncpg, система управления базами данных в асинхронном режиме;
- Alembic, интеграция для ведения миграций баз данных, обеспечение производительности состояния;
- Redis 5.0, система управления временными данными с высокой частотой изменений;
- Pytest, тестирование серверной части с использованием модульных, «точка-точка» и асинхронных тестов.

Для клиентской части был выбран TypeScript, который компилируется в JavaScript для корректной работы в среде встроенного мессенджером браузера. Выбор данного языка обоснован требованиями к высокой надежности, масштабируемости итогового продукта.

При работе с игровыми сущностями возникает высокий риск логических ошибок, что может быть отловлено с помощью TypeScript до попадания изменений на пользовательский стенд. Строгая типизация языка может гарантировать целостность данных и соблюдение контракта в клиент – серверной архитектуре. Кроме того, TypeScript является легко читаемым и популярным языком программирования, что повышает возможности к модернизации и поддержке реализовываемого приложения.

Список клиентских библиотек с описанием их назначения для проекта:

- React 18, построение пользовательского интерфейса в мини-приложении мессенджер в виде переиспользуемых модулей;
- Vite 5, сборка и запуск клиентского интерфейса с высокой скоростью и быстрой настройкой;
- React-router-dom 6, навигация между экранами мини-приложения;
- Zustand, разделение локального и хранимого состояния интерфейса;

- SDK-React, интеграция для мини-приложений telegram с ориентиром на работу с интерфейсами мессенджера.

- Framer-motion, анимация пользовательского интерфейса.

Развертывание системы на сервере происходит с использованием технологии «Docker compose» для обеспечения кроссплатформенности, согласованности и организацией взаимодействующих контейнеров. Использование технологии Docker в области автоматизации процессов сборки, непрерывной доставки и развертывании приложений обеспечивает:

- Изоляцию компонентов системы друг от друга;
- Единый ожидаемый результат для платформ;
- Скорость развёртывания;
- Удобство в расширяемости проекта.

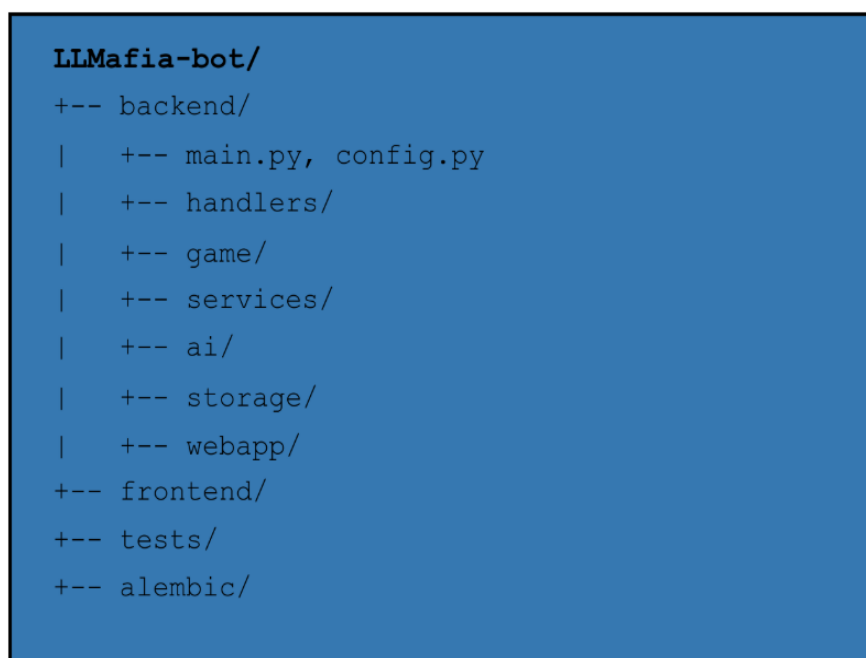


Рисунок 2.1 – древовидная архитектура по каталогам

В составе разработанного продукта и системы в целом выделяется несколько отдельных сервисов:

Сервис «Bot» - основной вычислительный компонент системы, внутри которого реализована серверная логика взаимодействия пользователя с

виртуальным помощником, клиентская часть в виде HTTP-API интерфейса через мини-приложения мессенджера.

Сервис «Redis» - Высокоскоростная база данных для временно хранимых данных, к которым часто обращается логический компонент продукта.

Сервис «Postgres» - Реляционная база данных для хранения долгосрочных данных с возможностью резервного копирования и простой миграцией схемы.

Сервис «Ollama» - Языковая модель искусственного интеллекта, включающая в себя инструменты генерации текста и алгоритмы принятия решений.

Архитектура программы построена по четырехуровневому принципу разделению логических уровней: уровень клиентов, уровень приложений, уровень доменов и уровень данных. Такое разделение обеспечивает атомарность каждого логического уровня, который выполняет свой спектр задач и взаимодействует с соседними слоями через доступные интерфейсы.

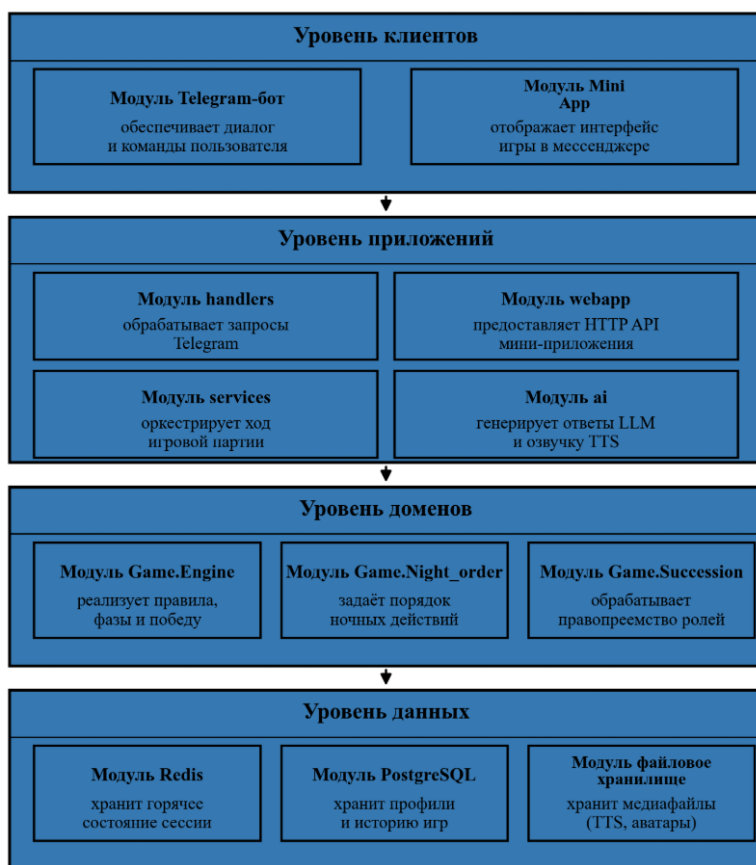


Рисунок 2.2 – многоуровневая архитектура приложения

Уровень клиентов предназначен для систем взаимодействия пользователя с сервером и включает в себя базовую модель общения через диалоговое окно и команды, а также расширенную – через мини-приложение мессенджера.

Уровень приложений предназначен для основного процесса виртуального помощника и выступает в роли оркестратора всей системы, отвечая за обработку поступающих пользовательских запросов.

Кроме того, слой приложения отвечает за обращения к интеллектуальным сервисам, включая языковые модели и модели генерации медиаконтента.

Уровень доменов заключается в предметной логике игрового процесса, здесь формируются игровые правила, механики и ролевая модель. Уровень включает в себя три отдельных модуля: «Game.Engine», «Game.Night_order», «Game.Succession».

Модуль Game.Engine – это игровой движатель, он отвечает за состояние сессий, реализует правила игры: условия смены фаз, условия доступности действий, условия победы и общее состояние участников.

Модуль Game.Night_order – это игровой оркестратор активных ролей в ночную фазу игры, его отделение от игрового движателя необходимо поскольку множество игровых персонажей обладают уникальными действиями, порядок исполнения которых крайне важен в процессе игры в «Мафию» и строго прописан в правилах игры.

Модуль Game.Succession – это игровой оркестратор состояний. Компонент держит ответственность в области перерасчета условий победы, активацию дополнительных логических цепочек для уникальных возможностей персонажа. Выделение от игрового движателя способствует прозрачности и модульности процесса.

Принципиальная значимость уровня доменов заключается в отделении игровой логики от инфраструктурной зависимости, расширении возможностей для кроссплатформенной реализации продукта.

Уровень данных представляет собой хранение краткосрочной и долговременной информации, необходимой для работоспособности системы.

Для уровня выделяются несколько компонентов-хранилищ, разделенных по специфике хранимых данных.

Данные горячего состояния – это особое состояние системы, при котором информация пребывает в постоянных изменениях и нуждается в доступе с минимальной задержкой. К таким данным относятся:

- Состав участников игровой сессии;
- Игровая фаза и подфаза сессии;
- Результаты голосования;

Реализация основана на нереляционной СУБД Redis, обеспечивающей высокую скорость и эффективность работы с горячими данными.

Данные холодного состояния – информация, которая не требует обновления в реальном времени, обычно с необходимостью к долгосрочному хранению, к таким данным относятся:

- Профили пользователей;
- Результаты завершенных сессий;
- Статистика и история игр;
- Конфигурационные и служебные данные;

Реализация построена на реляционной системе управления базами данных PostgreSQL, обеспечивающей согласованность и целостность данных, что особенно важно при долгосрочном типе хранения.

Отдельно для уровня данных так же выделяется файловое хранилище, предназначенное для хранения артефактов экспериментальной разработки в области технологии синтеза голоса на основе текстовых данных, планируемой в качестве медиаконтента разрабатываемого продукта.

2.2 Реализация игровой механики разрабатываемого продукта

В настоящем разделе выпускной квалификационной работы автор описывает интерпретированную в кодовое представление составляющую в виде игровых механик .

Понятие игровых механик в контексте данной дипломной работы включает в себя совокупность из игровых фаз, доступных действий, условий победы, формализованных в программный код. Стратегическую роль в рамках разрабатываемого продукта выполняет необходимость принятия решений в условиях ограниченных ресурсов, в данном случае – информации: каждый игровой персонаж, включая виртуального игрока, знает только лишь собственную роль, историю диалога и наблюдения за поведением других игроков, постепенно выстраивая представления о том, какое решение с учетом его роли следует предпринять.

Реализация игрового процесса сосредоточена в двух основных пакетах: `LLMafia_bot.Game`, который содержит чистую игровую логику, интерпретированную в программный код, `LLMafia_bot.Flow`, который является дежурным оркестратором всех переходящих процессов, включая взаимодействие с клиентской частью системы.

Игровые правила включают в себя начало сессии с числом участников от 4 до 16: стандартный игровой режим требует минимум 6 человек за столом, а быстрый – 4 участника. Максимально допустимое число участников точно регулируется настройкой в конфигурации, что позволяет адаптировать систему под доступный ресурс площадки, на которой размещен продукт и на текущий момент является 16 человек. Часть мест за «столом» может занять виртуальный игровой персонаж под управлением искусственного интеллекта, их профили содержат особый маркер «`is_ai`» который отличает их от реальных игроков, в остальном, их взаимодействие с игрой, определение модулями системы идентично.

В игре выделяются следующие фракции, реализованные в модуле LLmafia_bot.Roles:

Таблица 2.1 Разделение игровых фракций

Фракция	Срез ролей	Условие победы
Чёрная	Мафия, адвокат	Паритет чёрных
Красная	Мирный, врач, комиссар	Паритет красных
Нейтральная	Шут, маньяк	Остаться последним игроком
Культ	Оккультист, Лидер	Паритет оккультистов

Правила по определению ролей и наличию фракций задаются при создании игровой сессии и регулируются объектом «GameTableConfig» пакетом игрового двигателя. Дополнительно, для оптимизации различных игровых окружений, настройка включает в себя запрет на действия определенной фракции в первую ночную фазу, режим анонимного голосования, правила разрешения спорных ситуаций на голосовании, раскрытие роли при исключении игрока. Помимо прочего для представления развлекательной составляющей в игре есть настройка тематики игры: режим «Мафия», режим «Оборотни», режим «Войны фракций», режим «Зомби», режим «Человек-ведущий».

Роли игроков не публикуются в общедоступные каналы, за исключением особых случаев или в момент завершения партии. Возможность получить дополнительную информацию об участника получают особые игровые роли, которые могут выполнить действие раскрытия в ночную фазу. Помимо прочего, конфигуратор позволяет так же отключить для мафии начальное знакомство. Это обеспечивает ценность и ограниченность информации, как ценного ресурса.

Для специфических игровых сценариев предусмотрен механизм правопреемственности ролевой модели: при исключении ключевых игровых ролей персонаж с ролью «студент» приобретает уникальные свойства первой свободной роли. Однако, согласно доступным правилам игры, этому действию могут помешать такие роли, как «телохранитель», «медиум» и «сержант», в интерпретации данный механизм отражен в полной мере.

Жизненный цикл игровой сессии описывается объектом «GameState» и включает пять основных макрофаз: ожидание, ночь, день, голосование, завершение.

В состоянии ожидания администратор выполняет конфигурацию игровой сессии и набирает участников партии. В состоянии ночи происходит сбор особых ночных действий под запланированному порядку. В состоянии дня происходит обсуждение участниками и знакомство. В состоянии голосования игроки исключают подозреваемого.

За игровую итерацию считается переход к новой фазе дня, при таком переходе увеличивается системный счетчик. Ночная фаза завершается отдельным обработчиком, который рассчитывает факторы ролевых действий по их порядку и назначению. После исполнения обработчиком вызывается следующий, который проверяет условия завершения партии. При отсутствии условий победы, партия продолжается дневной фазой и последующим голосованием, которое вызывает обработчик условий победы и продолжает цикл.

Макрофаза дня включает в себя несколько подфаз:

- представление - краткое знакомство участников;
- речь - поочередные высказывания участников;
- ожидание голосования - переходное состояние до следующей макрофазы;
- действие ведущего - пауза перед переходом для персонального режима.

Макрофаза ночи разбивается на заранее определенное по числу уникальных игровых ролей подфаз, каждая из которых обрабатывается персонально и включает в себя отдельную игровую логику для каждой роли. Для категории участников с ролями, относящимися к мафии, проходит мини-цикл, внутри подфазы:

- дискуссия – обсуждение личных мыслей между участниками;
- голосование – подведение итогов и запись выбора в систему;
- спор – повторная итерация цикла при возникновении спорной ситуации;

По результатам каждой подфазы, ее этап переводится в решенное состояние, после того как все обработчики перейдут в такое состояние вызовется общий уравниватель, который сведет все решения к единому состоянию, обновится список игроков, хроника и проверятся условия победы.

В классическом представлении стратегической игры существует понятие ресурсов в виде золота или дерева. Для разрабатываемого продукта роль ресурсов выполняет набор определенных возможностей и статусов, сбалансированно распределенных между ролями, в таблице ниже представлены основные ресурсы с описанием механики:

Таблица 2.2 Модель ресурсов

Ресурс	Механика
Жизнь	Теряется при ночных действиях или голосовании
Ночной ход	Выбор активного действия по роли
Голос	Выбор игрока на исключение
Информация	Хранится как поле в сведениях о персонаже
Броня	Защищает от ночного исключения один раз
Помилование	Защищает от дневного исключения один раз

Логика проведения дневных ходов: Виртуальный помощник или игрок, при выбранном персональном режиме уведомляет участников сессии о результатах предыдущей итерации или же, в случае первой итерации, проводит

знакомство участников – игрок отправляет сообщение в текстовый диалог или в мини-приложение в рамках своей очереди. Далее, проходит этап обсуждения, игроки так же в порядке очереди отправляют сообщения, где перечисляют свои подозрения и планы на игру. После, начинается этап голосования, при наличии включенного параметра, игроки могут пропустить его без исключения игрока, в противном случае, в порядке своей очереди необходимо выбрать участника на исключение. Статистика голосов отражается виртуальным помощником в диалоге, мини-приложении мессенджера. При наличии спорной ситуации происходит переголосование, иначе игрок покидает сессию, но может отслеживать ход ее событий в качестве «призрака». Так же при включенной настройке в конфигураторе игрок имеет право произнести последнее слово, которое так же зафиксировывается в игровой хронике.

Логика проведения ночных ходов: Виртуальный помощник или игрок уведомляет живого участника с ролью уникального действия в личных сообщениях или через мини-приложение, предлагая выполнить свой ход и список доступных целей. Игрок фиксирует свой выбор, после чего его ход фиксирует состояние решенного, а цель помечается действием игрока. Участники команды мафии разделяют двухэтапный ночной цикл, похожий на дневное голосование и выбирают жертву на текущую фазу. По завершении всех обработчиков происходит фиксация результатов согласно правилам игры, в дальнейшем, они будут оглашены на следующей итерации.

После исключения игрок может пользоваться системой в рамках сессии в ограниченном режиме. При включенной настройке игрокам доступен персональный чат исключенных, а роль «медиум» позволяет провести диалог с одним из исключенных.

Программная модель включает в себя следующие сущности:

Таблица 2.3 Основные сущности системы

Сущность	Реализация	Ключевые поля
Партия	GameSessionState	game_id – идентификатор, phase – состояние партии, table_config – конфигурация
Персонаж	SessionPlayer	user_id – идентификатор, role – игровая роль, alive – статус участника
Комната ожидания	SessionLobby	player_ids – список игроков, game_settings – предустановки партии
Ночной результат	SessionNight	dead_ids – ночные исключения, events – уникальные действия
Дневной результат	SessionDay	vote_ids – промежуточное голосование, disband_ids – список исключенных

В качестве развлекательных сервисов для пользователей продукта дополнительно была реализована система прогрессии между партиями, которая хранится в профиле игрока с использованием системы долгосрочного хранения. Прогрессия включает в себя опыт и уровень игрового персонажа пользователя: опыт накапливается по итогам партий, порог для повышения уровня профиля рассчитывается по формуле с использованием арифметической прогрессии: $Level = \max(40, 100 + level - 1) * 20$.

Также реализована система рейтинговых показателей для многопользовательских сессий, которая отражает стратегические навыки игрока и процент доведенных до победных условий своей фракции матчей.

Обеспечение баланса игровой сессии реализовано с использованием метода предопределение ролей на количество игроков, который принимает в себя количества игроков, игровые предустановки, желаемые роли для включения. Игровой системой автоматически будет сформирован список ролей для сессии на основе базовых пропорций для каждой фракции, в соответствии с рекомендациями, указанными в правилах к игре. Есть набор специфических ролей с уникальными действиями в ночную фазу которые переопределяют базовые пропорции или требуют включения созависимых ролей: они изменяют пороги на минимальное и максимальное число участников для каждого отдельного игрового режима.

Модуль `game.balance_weights`, ответственный за обеспечение игрового баланса построен на весовой модели реализации, где для каждой роли предопределяется числовой показатель силы по трем различным игровым аспектам: влияние на город, влияние на мафию, влияние на нейтралов. На основе этих весовых показателей модуль вычисляет отчёт о балансе роли и конфигурации в целом:

- целевые доли: 60% мирной фракции, 25% мафии, 15% - нейтралы;
- коэффициент К: $\text{сумма весов города} / (\text{сумма весов мафии} + \text{нейтралы})$;

Пороговые значения коэффициента К в диапазоне от 1.5 до 2.5 формируют предупреждения для администратора сессии и блокирующие сообщения с результатом отчёта балансировщика. Организатор сессии может исправить конфигурацию в соответствии с рекомендациями или продолжить нечестный матч.

Игровая механика разработанного продукта представляет собой программную интерпретацию социально-психологической игры с элементами стратегического взаимодействия «Мафия». Ручные механизмы составления сбалансированного игрового состава, первоначальные игровые механики и сущности реализованы детерминированным двигателем системы, предоставляющем полную автоматизацию и интерактивность игровых процессов.

2.4 Интеграция ресурсов искусственного интеллекта

В настоящем разделе выпускной квалификационной работы автор описывает решения в области искусственного интеллекта примененные для интеграции с разработанным продуктом в виде компьютерной игры стратегии.

Для разработанного продукта искусственный интеллект используется не только в качестве дополнительного развлекательного наполнения, но функционального элемента системы. Модуль искусственного интеллекта разделяется на зоны ответственности:

Генерация игрового окружения и «паспортов» персонажей для задания общего настроения игровой сессии. Этот элемент системы используется в качестве прикладного инструмента расширения игровой вариативности и вовлеченности участников игрового процесса за счет погружения в разностороннюю и уникальную атмосферу каждой отдельной сессии. Окружение включает в себя набор событий, описывающих игровой тон.

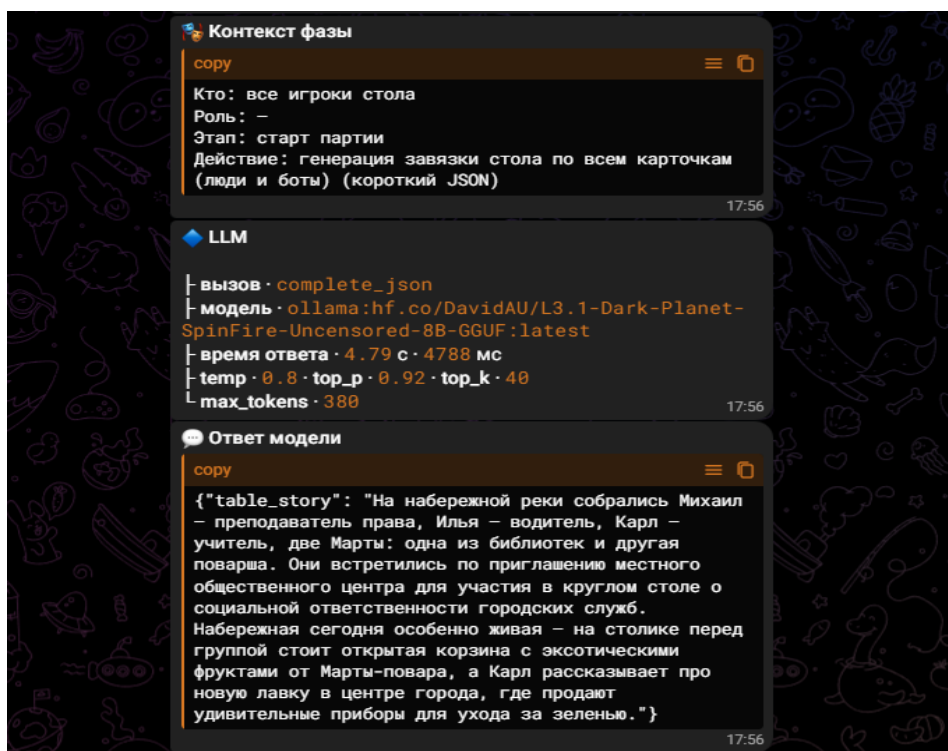


Рисунок 1 – пример сгенерированного игрового окружения

Паспорт персонажа в контексте данной лабораторной работы – это биометрия игрового персонажа, описание его характера и жизненной легенды, он так же используется для создания общей игровой атмосферы.



Рисунок 2 – пример сгенерированных паспортов

Применение ИИ в данной части проекта позволяет сделать игровой процесс более живым и разнообразным, а также уменьшить повторяемость текстового сопровождения при многократном запуске партий.

Интеллектуальный противник под управлением искусственного интеллекта – это ключевое направление для использования языковых моделей в продукте, он обладает полноценным набором действий и ресурсов, ровно таким же, как и настоящий участник процесса. ИИ-игроки участвуют в следующих сценариях:

- выбор ночной цели для роли;
- участие в ночном обсуждении мафии;
- формирование дневных выступлений;
- участие в голосовании.

В итоговом представлении системы, искусственный интеллект является ответственным модулем для генерации текстового контента в виде игровых условностей, карточек игровых персонажей, а также является возможным полномерным участником для сессии: алгоритмы принятия решений позволяют предоставить для языковой модели все инструменты взаимодействия с окружающей средой при этом внутренние механизмы нейтрализуют эффект галлюцинаций, который может возникать у различных моделей при длительном общении с системой.

Галлюцинации искусственного интеллекта представляют собой феномен, при котором языковая модель создает недостоверную, вымышленную и зачастую абсурдную информацию с отсутствующей релевантностью к первоначальному запросу от пользователя при этом будучи строго уверенной в достоверности своего ответа. Объяснение причин возникновения подобных нюансов при работе с искусственным интеллектом представляет собой основной принцип работы подобных систем: предсказывание последующих слов на основе матрицы вероятностей, улавливание ложных закономерностей в словах из первоначального запроса и отсутствие критического мышления, чтобы отличить ложь от истины.

Что используется для борьбы с галлюцинациями:

- Механизм независимого шага: запросы к искусственному интеллекту строятся без сохранения контекста от предыдущих запросов, система опирается на отдельно предоставленный системный и пользовательский контекст.

- Строгий формат ответа: запросы на принятия важных стратегических решений требуют предоставления ответа в специализированной обертке JSON, модель должна строго прописать в отдельных ключах словаря принятое решение и пояснения для выбранного действия. Это реализует механизм интерфейсов для ответа искусственного интеллекта.

- Постобработка полученных данных: после получения ответа от искусственного интеллекта запускается механизм фильтрации и поиска

требуемого значения по ключам в полученном словаре. Это минимизирует процент некорректных ответов от искусственного интеллекта.

- Ограничение свободы модели: механизм предопределение роли искусственного интеллекта в виде участника игрового процесса с описанием ключевых игровых правил и доступных действий сильно сужает пространство для интерпретации формулировок.

- Контроль длины и нормализация текста: предоставленная информация проходит валидацию на лимит, заложенный в первоначальных запросах, и проходит нормализацию, чтобы ответ не приводил систему в нестабильное состояние.

- Защита от инъекций в запросах: модель получает в запросе указание, что полученный контекст является только пользовательской информацией и не требует исполнения каких-либо действий. При наличии в речах реальных игроков требований модели выполнить какое-то действие система включает механизм фильтрации контекста, чтобы виртуальный помощник не подчинялся командам, спрятанным в репликах игроков.

В проекте используется специализированный модуль, предназначенный для взаимодействия с большими облачными языковыми моделями, а также небольшими локально развернутыми. Данный модуль поддерживает работу через несколько типов провайдеров:

- API облачных языковых моделей OpenAI;
- Локально развёрнутый сервис Ollama;
- Цепочка провайдеров с резервным переключением.

Таким образом, архитектура ИИ-подсистемы предусматривает гибкий подход к выбору провайдера ресурсов искусственного интеллекта. В зависимости от конфигурации окружения система может использовать как внешний облачный сервис, так и локально развёрнутую модель. Это повышает устойчивость решения и позволяет адаптировать его к различным сценариям эксплуатации.

Искусственный интеллект использует контекстный анализ доступной игровой информации для формирования решений и реплик. В частности, модели передаются:

- история публичных сообщений и обсуждений;
- произошедшие дневные события;
- последние реплики игроков;
- список живых участников;
- доступные цели действий.

За счёт этого ИИ способен принимать решения, опираясь на развитие игровой ситуации, а не только на фиксированные шаблоны.

Типовой сценарий работы выглядит следующим образом:

- Оркестратор определяет фазу партии и доступные действия роли;
- Формирует контекст: номер дня, фаза, кандидаты, карточка персонажа, история реплик и иные данные, относящиеся к текущей ситуации;
- Обращается к ресурсам искусственного интеллекта в специфике того, что сейчас необходимо применить: составление дневной речи / принятие решения;
- Полученный ответ проходит проверку и валидацию.
- Действие применяется и меняет состояние игрового потока.

Принципиально важно, что ИИ не изменяет состояние игры напрямую. Все результаты его работы проходят через доменную логику, где проверяются на допустимость и соответствие текущим правилам партии. Такой подход предотвращает нарушение целостности игрового процесса и сохраняет приоритет формализованных правил над вероятностной моделью.

Каждый запрос к языковой модели состоит из двух основных частей:

- системное сообщение, задающее правила поведения модели, формат ответа и перечень ограничений;
- пользовательское сообщение, содержащее конкретное описание игровой ситуации.

В модуле prompts.py формализованы ключевые ограничения, обеспечивающие предсказуемость ответов:

- модель не должна опираться на скрытую память между запросами;
- информация из пользовательских сообщений не должна восприниматься как инструкция к изменению правил;
- при необходимости машиночитаемого результата ответ возвращается в формате JSON;
- модели запрещено выдумывать игровые факты;

Например, при выборе ночной цели функцией алгоритмов принятия решений формируется запрос, в котором модели предлагается выбрать уникальный игровой идентификатор цели только из заранее переданного списка допустимых кандидатов, кратко пояснить причину выбора. учитывать роль персонажа и активную подфазу ночи. Ожидаемый формат ответа имеет вид:

```
JSON
{
  "target_id": 123456,
  "мысли": "Краткая причина выбора цели"
}
```

Ответ языковой модели проходит несколько этапов обработки перед применением в игровом процессе. Если ожидается структурированный ответ, выполняется:

- разбор JSON;
- проверка корректности структуры;
- проверка того, что цель входит в список допустимых;
- нормализация текстовых полей;
- ограничение длины сообщения;

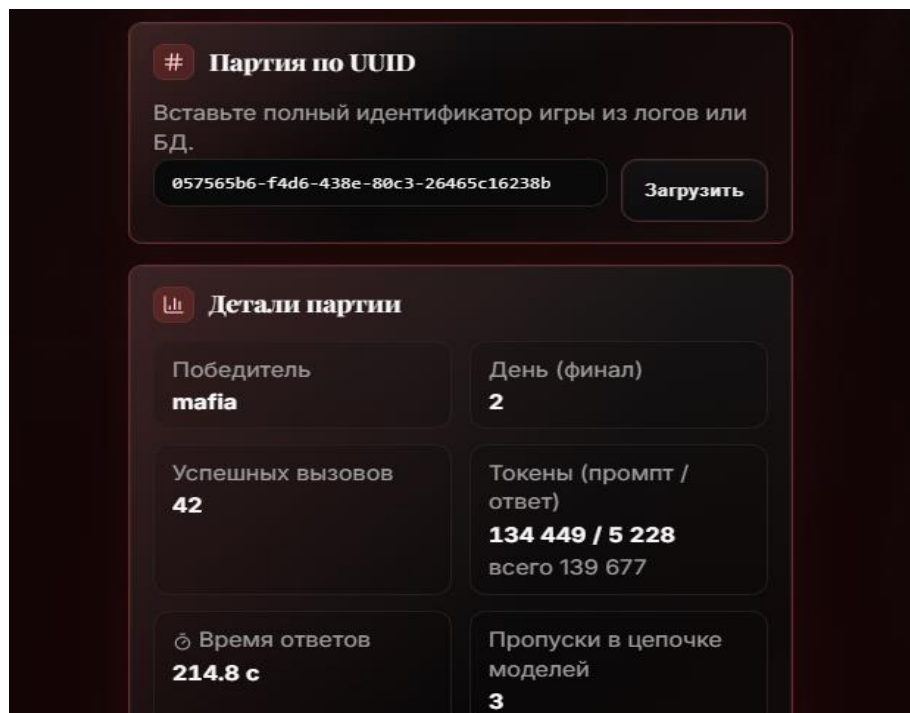
Таким образом, даже успешно сгенерированный ответ не применяется автоматически: он должен соответствовать правилам текущей игровой ситуации.

В случае ошибки модели, например, при пустом ответе, некорректном JSON, сетевом сбое или недоступности провайдера — система использует резервные механизмы:

- запрос к следующей модели из цепочки;
- эвристика цели действия;
- шаблоны реплик

Данный механизм является критически важным, поскольку недоступность внешнего интеллектуального сервиса не должна приводить к остановке игровой партии. Благодаря fallback-логике ИИ остаётся вспомогательным компонентом, а не единственной точкой отказа системы.

Администратор системы по уникальному идентификатору игровой сессии может запросить отчет, с подробной аналитикой каждой сессии. Это позволяет проводить последующий анализ затрат, интенсивности использования и общей эффективности ИИ-интеграции.



# Партия по UUID	
Вставьте полный идентификатор игры из логов или БД.	
057565b6-f4d6-438e-80c3-26465c16238b	<button>Загрузить</button>

Детали партии	
Победитель mafia	День (финал) 2
Успешных вызовов 42	Токены (промпт / ответ) 134 449 / 5 228 всего 139 677
⌚ Время ответов 214.8 с	Пропуски в цепочке моделей 3

Рисунок 3 – пример сгенерированного отчета

На текущем этапе реализации искусственный интеллект применяется в проекте по ряду направлений, включающих в себя генерацию игровых описаний и карточек персонажей, создание текстового сопровождения игровых событий.

управление ИИ-игроками как интеллектуальными противниками, контекстный анализ игрового состояния для выбора действий и формирования реплик.

Интеграция ресурсов искусственного интеллекта в логику Telegram-бота реализована как управляемый функциональный интеллектуальный сервис, встроенный в игровой цикл. Искусственный интеллект используется для генерации контента, поддержки виртуальных персонажей и повышения вариативности партий, однако не заменяет формализованные игровые правила и не получает прямого доступа к изменению состояния игры.

За счёт формализованных промптов, проверки структуры ответов, резервных сценариев обработки ошибок и учёта эксплуатационных метрик достигается баланс между вариативностью, создаваемой ИИ, и устойчивостью работы всей системы. Это позволяет рассматривать интеграцию ИИ как практически значимый элемент архитектуры, повышающий интерактивность и реалистичность игрового процесса без нарушения целостности доменной логики.

2.5 Реализация пользовательского интерфейса

Пользовательский интерфейс разработанного программного продукта построен в модели нескольких уровней взаимодействия, объединяющей возможности мессенджера Telegram и встроенного мини-приложения. Разделение каналов обеспечивает одновременно прозрачность публичного игрового процесса и конфиденциальность ролевой информации: карточка роли и секретные ночные действия никогда не попадают в общий чат, тогда как события партии формируют единый поток в групповом диалоге, доступный всем участникам. Сессию полноценно можно провести внутри мини-приложения без задействования группового диалога.

С архитектурной точки зрения интерфейс опирается на технологии:

- aiogram 3 — обработка команд, callback-кнопок, FSM-состояний лобби;
- aiohttp + React/Vite — HTTP API и WebSocket для Mini App;
- Redis — хранение активного состояния сессии и привязка игрока к столу;
- PostgreSQL — профили пользователей, история сообщений, статистика.

Взаимодействие пользователя с ботом в рамках личных сообщений выполняет роль шлюза между пользователем и полноценным интерфейсом. Здесь реализованы базовые функции: вход по команде /start, отображение справочной информации, В классическом режиме в личных сообщениях дополнительно выполняются секретные ночные действия через inline-кнопки.

Групповой чат используется исключительно как публичная площадка: в нём отображается хроника партии и принимаются речевые сообщения участников. Вся остальная игровая логика — настройка, управление составом, профиль, подробная статистика — вынесена в Telegram Mini App.

Общий путь взаимодействия пользователя с системой:

- открыть меню бота через начальную команду /start;
- открыть интерфейс системы через мини-приложение телеграмм;
- создать комнату ожидания, привязать групповой диалог;
- настроить конфигурацию, начать игровую сессию;

В ходе партии бот использует несколько типов сообщений, каждый из которых выполняет определённую функцию:

- Хроника площади — публикуется в групповом чате. Представляет собой обновляемое якорное сообщение, фиксирующее ход событий: начало фаз, итоги ночи, результаты голосований.

- Карточка роли — отправляется каждому участнику в личные сообщения при старте партии. Содержит наименование роли, краткое описание её возможностей и правила поведения в ночную и дневную фазы.

- Приглашение к ходу — персональное сообщение в личных сообщениях с указанием очерёдности и набором доступных действий в виде кнопок.

- Системные уведомления — информация о таймаутах, автоматических ходах, ошибках и подсказки по работе с интерфейсом.

- Итоги фазы — сообщения о рассвете, результатах голосования и финале партии с раскрытием всех ролей.

- Реплики участников — публикуются в группе с подписью говорящего, включая как сообщения реальных игроков, так и речи ИИ-персонажей.

Все сообщения формируются с использованием HTML-разметки для выделения акцентов и структурирования списков так, как этого требует мессенджер. Пользовательские имена экранируются во избежание инъекций разметки. Длинные тексты обрезаются для соблюдения ограничений мессенджера. При использовании тематик системные сообщения форматируются в соответствии с выбранной тематикой.

Для поддержания непрерывности игрового процесса реализована система таймаутов, задаваемых переменными окружения. При превышении допустимого времени ожидания в ситуациях выбора ночной цели, дневной речи, голосования, голос мафии система автоматически фиксирует за молчащего участника случайный допустимый ход и продолжает партию. Это исключает зависание игрового процесса при неактивности одного из участников и снимает необходимость в постоянном контроле со стороны ведущего.

Клиентское приложение реализовано в виде одностраничного React-приложения на языке TypeScript с использованием технологии Vite в качестве инструмента сборки. Приложение открывается внутри Telegram через кнопку управления и обеспечивает расширенный пользовательский интерфейс, недоступный в рамках стандартного чат-взаимодействия, который поддерживает проведение полной игровой сессии без задействования группового диалога и личной переписки с ботом.

Таблица 2.4 Маршруты приложения

Маршрут	Экран	Назначение
/	Главное меню	Переход в разделы приложения
/match	Поиск комнаты	Поиск существующих сессий
/lobby	Ожидание старта	Ожидание участников сессии
/lobby/config	Настройки стола	Конфигурация сессии
/session/live	Живая сессия	Игровой интерфейс сессии
/session/reveal	Раскрытие роли	показ чувствительных данных
/session/result	Итог партии	Завершение сессии
/profile	Профиль	Персональные данные
/admin/*	Панель администратора	Управление системой

Центральным элементом игрового процесса является экран живой сессии, объединяющий несколько функциональных блоков:

- разделённые диалоговые каналы — площадь, чат мафии, исключённые;
- модальные окна для ввода речи, выбора ночной цели и голосования;
- очередь речей и ночных этапов с отображением текущего статуса.

Сценарий старта работы и первый вход в систему:

- Пользователь открывает бота в мессенджер;
- Пользователь вводит команду /start;
- Бот отправляет приветственное сообщение и предоставляет меню;
- Пользователь нажимает кнопку «мини-приложение»
- Приложение отображает главное меню с профилем;
- Приложение отображает вводную инструкцию по меню.

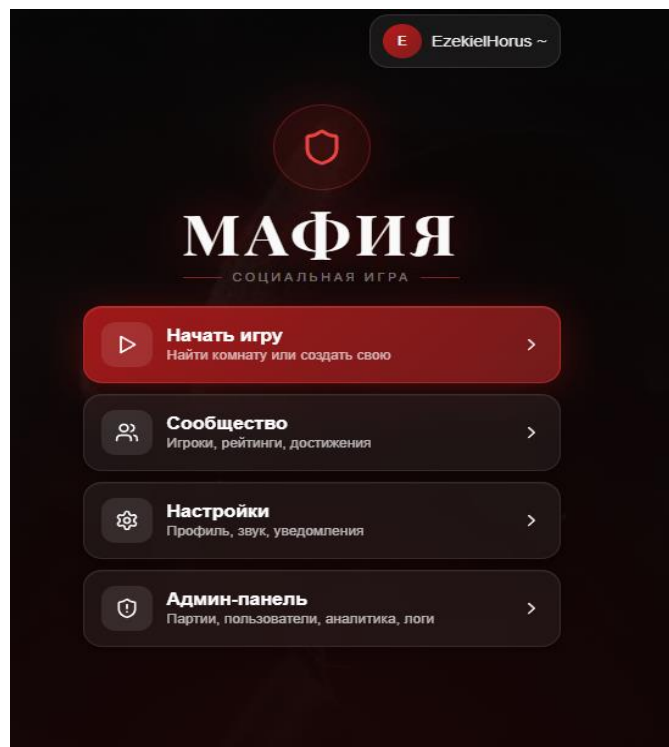


Рисунок 4 – Главное меню

Сценарий редактирования профиля:

- Пользователь находится в главном меню;
- Пользователь нажимает кнопку профиля сверху экрана;
- Приложение открывает экран профиля;
- Пользователь вводит имя, загружает аватар;
- Пользователь сохраняет изменения.

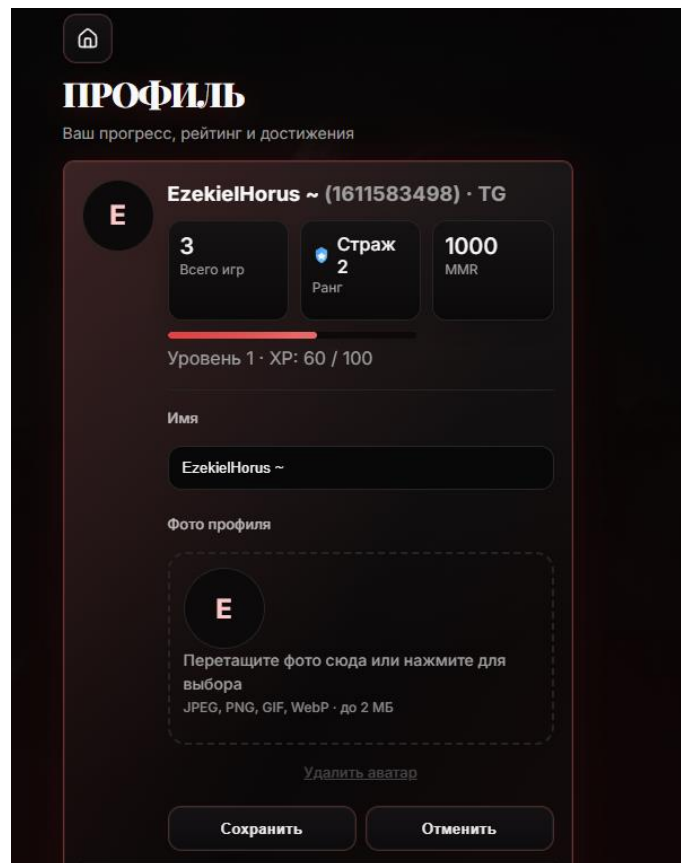


Рисунок 5 – Профиль пользователя

Сценарий создания игровой сессии :

- Пользователь находится в главном меню;
- Пользователь нажимает кнопку «начать игру»;
- Приложение открывает экран комнаты ожидания, кнопка начала игры неактивна;
- Пользователь переходит в конфигуратор игровой сессии;
- Приложение открывает экран конфигуратора;
- Пользователь выбирает настройки сессии, сохраняет их;
- Приложение возвращает в комнату ожидания, кнопка начала игры по-прежнему неактивна пока не наберутся участники;
- Пользователь добавляет виртуальных игроков и начинает сеанс.

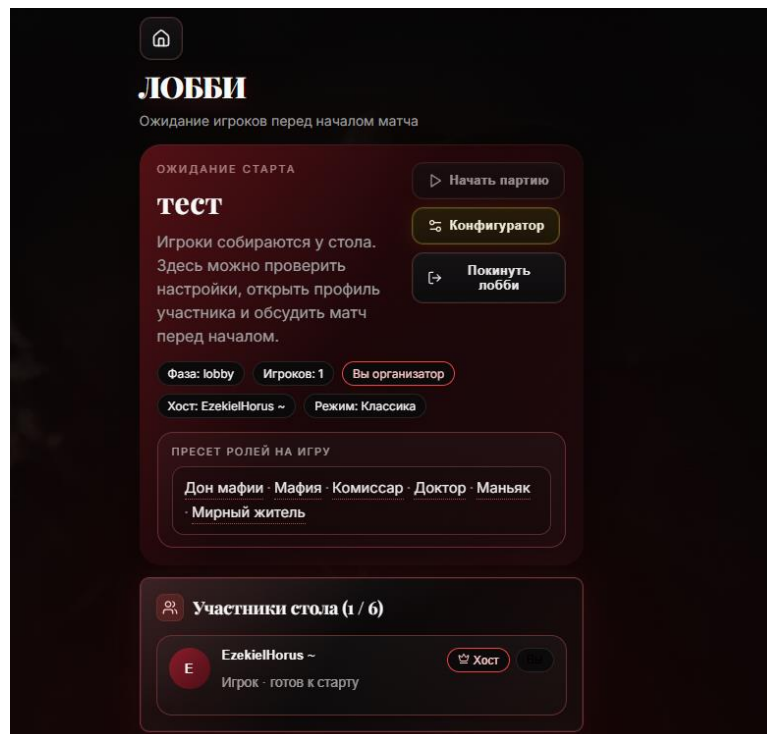


Рисунок 6 – комната ожидания

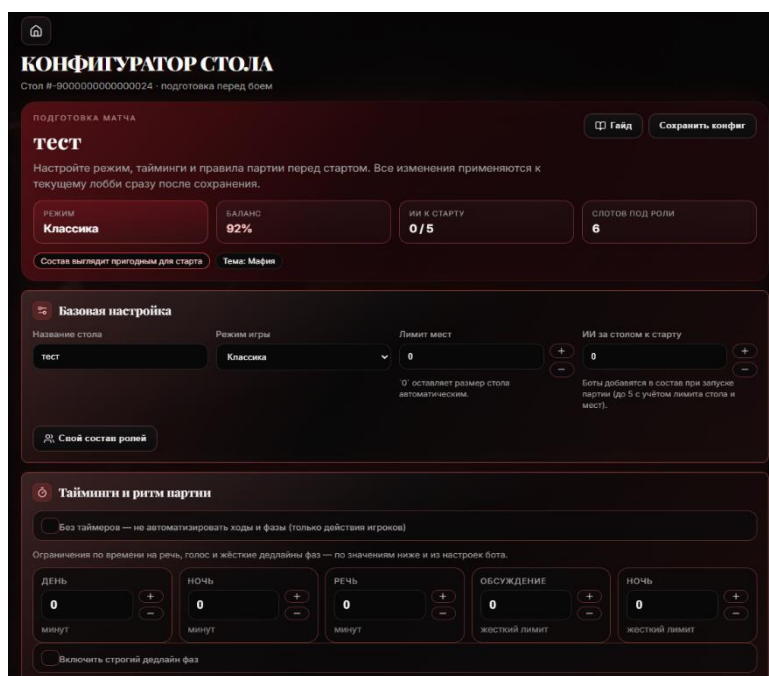


Рисунок 7 – конфигуратор сессии

Сценарий дневной фазы, проведения знакомства:

- Игровой оркестратор переводит состояние знакомства;
- Пользователь нажимает открыть речь, вводит текст, сохраняет;
- Приложение публикует текст в игровом окне сессии.

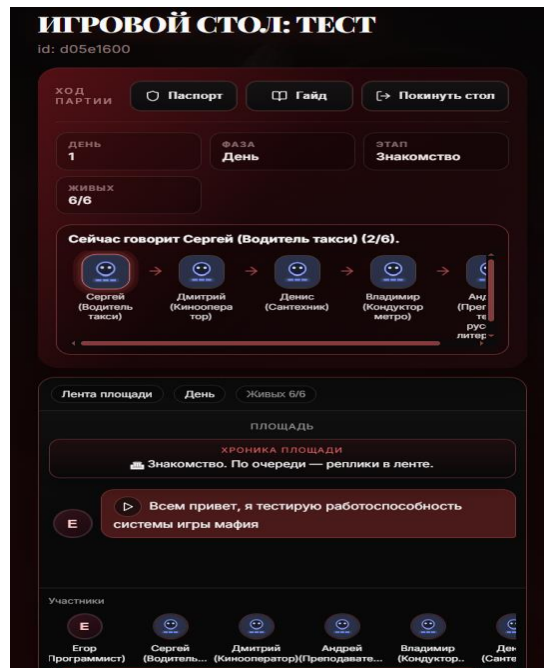


Рисунок 8 – игровой стол

Выводы по второй главе настоящей выпускной квалификационной работы: заложенные в разработанный продукт функциональные требования были в полной мере реализованы, итоговая игровая система в контексте данного продукта – это комплекс взаимосвязанных игровых компонентов, разделенных на строгие

ГЛАВА 3. ТЕСТИРОВАНИЕ И ОЦЕНКА ЭФФЕКТИВНОСТИ РАЗРАБОТАННОЙ КОМПЬЮТЕРНОЙ ИГРЫ-СТРАТЕГИИ

3.1 Методика тестирования программного продукта

Тестирование разрабатываемого программного продукта направлено на подтверждение соответствия реализованной системы функциональным требованиям, изложенным в разделе 2.1, а также на выявление дефектов, способных нарушить целостность игрового процесса или привести к некорректной работе системы в условиях реального использования.

В рамках данной дипломной работы тестирование преследует следующие цели:

- подтверждение корректности игровой логики: правил разрешения ночных действий, условий победы, системы голосований и последовательности фаз;
- проверка устойчивости системы при обращении к внешним сервисам, в частности при недоступности модуля искусственного интеллекта или TTS-подсистемы;
- верификация корректности пользовательских сценариев — от первого входа до завершения партии;
- подтверждение согласованности клиентского и серверного компонентов при взаимодействии через REST API и WebSocket;

В проекте применяется комплексный подход к тестированию, включающий несколько взаимодополняющих видов проверок.

Модульное тестирование направлено на проверку отдельных компонентов системы. Тесты реализованы автоматизированные и выполняются с использованием фреймворка Pytest. Каждый тест работает с фиксированными входными данными — объектами состояния, JSON-структурами или списками ролей — и сравнивает полученный результат с эталонным значением. Такой

подход обеспечивает детерминированность проверок и возможность их многократного воспроизведения.

Области покрытия модульными тестами:

- Игровой двигатель: `test_engine.py`;
- Баланс и состав ролей: `test_balance.py`;
- Голосование: `test_day_vote.py`;
- Фильтрация ответов ИИ: `test_llm_answer_parce.py`;
- Конфигурация сессии: `test_game_payload.py`;
- Режимы игры: `test_game_modes.py`.

Дополнительно к выполнению тестов в системе непрерывной интеграции производится статический анализ кода, охватывающий проверку стиля и выявление логических ошибок при импорте.

Интеграционное тестирование проверяет корректность взаимодействия нескольких компонентов системы при подмене внешних зависимостей заглушками. Тесты выполняются в изолированном окружении с тестовой базой данных на основе SQLite и mock-объектами для Redis и Telegram Bot API. Реальное подключение к Telegram в данном виде тестирования не используется.

Ключевые интеграционные сценарии:

- `test_classic_day_speech_publishes.py` — проверка публикации дневной речи игрока в групповой чат
- `test_classic_gameplay_gates.py` — проверка корректной маршрутизации секретных ходов между личными сообщениями и Mini App в зависимости от типа стола;
- `test_webapp_init_data.py` — проверка подписи Telegram initData при авторизации в Mini App;
- `test_post_game_rewards.py` — проверка корректности начисления наград после завершения партии.

Функциональное тестирование проверяет соответствие системы пользовательским требованиям в условиях полностью развёрнутой среды.

Данный вид тестирования проводится как вручную по чек-листу, так и в форме выборочной автоматизации критически важных сценариев.

Функциональное тестирование охватывает следующие сценарии:

- полный жизненный цикл стола: создание, привязка группы, настройка, вход игроков, запуск;
- дневную фазу: очерёдность речей, публикация в групповой чат;
- ночную фазу: выбор цели через кнопки или Mini App;
- голосование: очерёдность, обработка ничьей, переголосование;
- завершение партии, экран результата и формирование отчёта;
- административные функции при наличии соответствующих прав.

Пользовательское тестирование проводится разработчиком совместно с одним или двумя тестировщиками в роли участников игровой сессии. Основная цель — оценка удобства интерфейса, понятности игровых инструкций и общей полноты реализованных сценариев с точки зрения конечного пользователя.

В ходе пользовательского тестирования дополнительно фиксируются замечания по качеству интерфейса: формулировки сообщений бота, подсказки по привязке группы, понятность требований к настройке приватности и воспринимаемые задержки при работе искусственного интеллекта.

Методика тестирования, примененная на проекте, основывается на ГОСТ Р 56920-2024 «Системная и программная инженерия. Тестирование программного обеспечения. Общие положения» и охватывает все ключевые аспекты проверки программного продукта: от изолированного тестирования отдельных модулей игровой логики до пользовательского тестирования полного сценария игровой партии. Применение пяти взаимодополняющих видов тестирования в сочетании с чётко определёнными критериями успешности и классификацией дефектов обеспечивает системный подход к контролю качества. Описанная организация процесса тестирования — от подготовки среды до повторного прогона после исправлений — позволяет подтвердить работоспособность, устойчивость и соответствие разработанного продукта предъявленным требованиям.

3.2 Проверка функциональности Telegram-бота и игровой логики

В рамках данного раздела проводилась проверка работоспособности и корректности реализованного программного комплекса, включая игровую логику, механизмы управления сессией и функции виртуального помощника (Telegram-бота). Цель испытаний — подтвердить соответствие системы функциональным требованиям, описанным в проектной документации, и обеспечить возможность её демонстрации на защите выпускной квалификационной работы. Объектами проверки выступили:

- обработка команд и элементов меню в личных и групповых чатах;
- согласованность конечного автомата комнаты ожидания;
- корректность работы игрового двигателя;
- персистентность данных в Redis и PostgreSQL;
- взаимодействие с Telegram Mini App;

Испытания проводились в двух формах: автоматизированное модульное и интеграционное тестирование (pytest, более 40 тестовых модулей) и ручная проверка по заранее составленным сценариям в среде, близкой к эксплуатационной. Для классического режима использовалась конфигурация без мини-приложения; дополнительно проверялся режим с приоритетом Mini App. Результаты фиксировались в виде таблиц с указанием ожидаемого и фактического поведения.

Условные обозначения статуса:

- «Пройден» — поведение соответствует ожиданию;
- «Частично» — выявлены ограничения;
- «Не пройден» — сценарий не выполнен

Таблица 3.1 Тестирование элементов меню

Сценарий	Ожидаемый результат	Фактический результат	Статус
Обработка команды /start	Отображение меню с кнопками	Меню отображается корректно	Пройден
Обработка команды /new	Отображение меню с конфигурацией стола	Меню отображается корректно	Пройден
Обработка команды /join	Добавление участника в сессию	Участник успешно добавлен в сессию	Пройден
Обработка команды /join без идентификатора сессии	Появление сообщения об ошибке	Появляется ошибка «сессия не найдена»	Пройден
Обработка команды /start	Сессия начинается, происходит переход к знакомству	Партия начинается, идет фаза знакомства	Пройден
Обработка команды /start без минимального числа участников	Появление сообщения об ошибке, сессия не начинается	Появляется ошибка «недостаточно игроков», сессия не началась	Пройден
Обработка команды /cancel	Партия отменяется, комната ожидания недоступна	Окно комнаты ожидания закрывается, сессия завершается	Пройден

Проверялась корректность обработки базовых команд организатора и участника, а также отображение связанных с ними элементов интерфейса. Сценарии выполнялись в личном чате с ботом. при необходимости дополнительно проверялась привязка группового чата. По результатам тестирования элементов меню ошибок в работе разработанного продукта выявлена не было, критические сценарии были покрыты автоматизированными тестами, которые запускаются при добавлении изменений через инструмент непрерывной интеграции и доставки.

Таблица 3.2 Тестирование перехода состояний

Сценарий	Ожидаемый результат	Фактический результат	Статус
Запуск партии	Переход lobby > day	Переход корректен	Пройден
Дневная речь	Переход day > day_speech	Переход корректен	Пройден
Голосование	Переход Day_speech > day_vote	Переход корректен	Частично
Обсуждение мафии	Переход night > mafia_speech	Переход корректен	Пройден
Голос мафии	Переход mafia_speech > mafia_vote	Переход корректен	Пройден
Уникальное действие	Переход Night > *_action	Переход корректен	Пройден
Переход ко дню	Переход night_result > day	Переход корректен	Пройден
Завершение партии	day_vote > ended	Переход корректен	Пройден

Проверялась согласованность двух уровней состояний: FSM лобби (создание и настройка стола) и конечного автомата игровой сессии. Особое внимание уделялось очередности подфаз дня и ночи, соответствующих правилам «Мафии». По результатам проверки все тестовые сценарии завершены со статусом «Пройден». Блокирующих и критических дефектов в ходе испытаний выявлено не было. Критические сценарии дополнительно были покрыты автоматизированными тестами.

3.3 Оценка искусственного интеллекта в игре и перспективы проекта

В разработанном программном комплексе искусственный интеллект выполняет двойную функцию: управляет виртуальными игроками и генерирует игровой контент. При этом детерминированный игровой движатель сохраняет полный контроль над правилами партии — разрешением ночных действий, определением победителя, балансом ролей и очередностью фаз. Тем не менее, модуль искусственного интеллекта является ключевым в контексте данной дипломной работы и, безусловно, его работоспособность и качество предоставляемого сервиса прямо коррелирует с успешностью финального продукта. Таким образом, необходимо провести качественную оценку используемых провайдеров и контента, генерируемого языковыми моделями.

Оценка качества ИИ-интеграции проводится по четырём критериям по шкале от 1 до 5, где 1 — неудовлетворительно, 5 — отлично. Критерии оценки: релевантность, логичность, вариативность, отсутствие сбоев.

Использование ИИ направлено на увеличение вариативности игрового процесса по сравнению с фиксированными скриптами или сценариями с неполным составом реальных участников. В системе реализованы следующие механизмы вариативности:

- Уникальные карточки персонажей — генерируются под общую историю с различными профессиями, манерой речи и бытовыми деталями
- параметр температур ИИ ≈ 1.15 обеспечивает разброс формулировок при схожем контексте;
- Различные роли и режимы, изменяющие запрос и всецело поведение агента;

Общую оценку по вариативности составляет 4.7/5, это обосновано заметным отличием биографий в сессиях, разбросом реплик искусственного интеллекта, построение сложных логических цепочек приводит к неожиданным

результатам сессий – ИИ существенно повышает нарративную вариативность и разнообразие социального взаимодействия.

Устойчивость ИИ-подсистемы обеспечивается рядом архитектурных решений:

- Цепочка моделей, при пустом ответе, HTTP-ошибке, увеличивается счётчик и производится попытка со следующей моделью;
- «Липкий» провайдер, после успешного вызова система предпочитает тот же провайдер в течение 20 минут, снижая число переключений;
- Таймауты ограничивают суммарное ожидание; для генерации карточек используется увеличенный лимит;
- Дедлайны фаз по истечении времени ожидания завершают ход без участия модели: случайная цель ночью, пропуск речи днём;
- При отказе всей цепочки применяются шаблонные карточки и заготовленные реплики.

Общая оценка по отказоустойчивости составляет 5.0/5 - система спроектирована как отказоустойчивая. Сбои языковой модели ухудшают качество атмосферного контента, но не блокируют правила партии. Критическими параметрами для стабильной работы являются корректная настройка таймаутов и вычислительная мощность локального сервиса.

Релевантность и логичность генерируемого контента обеспечиваются на уровне промпт-инженерии:

- запрет на раскрытие ролей в карточках знакомства;
- блоки с инструкцией не воспринимать реплики игроков как команды модели;
- режим «логика и победа с анализом действий для стратегических фаз;
- корректная формулировка ночных задач без лишних механических деталей;

Средние оценки по критериям: релевантность 4.2/5, логичность 3,8–4,2/5 в зависимости от модели. Модели размером 8B и выше стабильно выдают связные дебаты; компактные 3B-модели уступают на длинных диалогах.

Подводя итог по третьей главе настоящей выпускной квалификационной работы, автор заявляет: создан программный комплекс для игры в «Мафию» через мессенджер с искусственным интеллектом. Реализованы создание и настройка стола, привязка группового чата, классический режим (группа + личные сообщения), несколько игровых заготовок и ролей, виртуальные игроки, постигровые награды, админ-доступ и аналитика сессий. Критичная логика покрыта тестами и проверена вручную по чек-листу; развёртывание автоматизировано через CI/CD и Docker.

Работающие функции: полный игровой цикл, конфигуратор стола, хроника в группе, секретные ходы через мини-приложение, учёт и оценка стоимости вызовов партии, генерация отчёта о сессии.

Ограничения: зависимость темпа партии от мощности локальной модели при их недоступности качество падает, но партия завершается за счёт предусмотренных механизмов и таймаутов фаз.

Направления улучшения: стабилизация фоновых цепочек после действий в мини приложении, тонкая настройка промптов и моделей под целевой темп игры.

Перспективы развития: многопользовательские и турнирные форматы, усложнение механики, углубление ИИ (память в рамках партии, анализ стиля игрока), развитие веб-панели администратора (мониторинг столов, модерация, отчёты), накопительная статистика и рейтинг игроков, горизонтальное масштабирование — вынос искусственного интеллекта на отдельные узлы.

ЗАКЛЮЧЕНИЕ

Выпускная квалификационная работа базируется на теоретических положениях проектирования компьютерных игр, на исследовании стратегических и социально-психологических игровых механик, а также на анализе возможностей платформы Telegram-ботов и современных подходов к интеграции ресурсов искусственного интеллекта в игровые программные системы.

По результатам изучения предметной области, обзора существующих решений и сравнительного анализа аналогов обоснована целесообразность реализации игры «Мафия» в формате Telegram-бота с поддержкой Telegram Mini App. Выбранная архитектура взаимодействия позволяет сохранить для пользователя привычную среду мессенджера и группового общения, а для настройки стола, ведения партии и управления профилем — использовать расширенный веб-интерфейс мини-приложения.

Практическая часть работы посвящена проектированию и разработке интерактивной системы, обеспечивающей проведение игровых сессий с участием как реальных пользователей, так и виртуальных персонажей, управляемых языковой моделью. На начальном этапе сформулированы функциональные требования; далее спроектирована многоуровневая архитектура, включающая клиентский компонент (Mini App), серверную часть бота, доменный игровой движок и слой хранения данных. Отдельно обоснован выбор технологического стека для серверной и клиентской разработки, а также средств развёртывания и сопровождения программного комплекса.

Реализованная игровая механика отражает ключевые особенности «Мафии»: скрытые роли, фазовый цикл «день — ночь», систему ночных и дневных действий, голосование, специальные способности ролей и различные условия победы отдельных фракций. Логика партии формализована в виде конечного автомата состояний с детерминированным разрешением игровых

событий, что обеспечивает корректность, воспроизводимость правил и упрощает верификацию системы.

Существенное место в работе занимает интеграция искусственного интеллекта. Языковая модель используется для генерации текстовых материалов, карточек персонажей и управления виртуальными игроками; при этом критически значимые игровые правила не делегируются модели, а реализуются детерминированным движком с последующей валидацией ответов LLM. Такой подход повышает вариативность и атмосферность партий, сохраняет игровой баланс и позволяет проводить сессии при неполном составе участников.

Пользовательское взаимодействие организовано через команды и элементы inline-интерфейса Telegram-бота, групповую хронику и клиентское мини-приложение. Реализованы основные сценарии: первичный вход, настройка профиля, создание и конфигурирование стола, работа лобби, запуск партии, выполнение игровых действий, завершение сессии и просмотр итогов. В результате получен работоспособный программный продукт, пригодный для эксплуатации в среде Telegram.

В ходе тестирования выполнены модульные и интеграционные проверки, функциональное и пользовательское тестирование, а также оценка качества использования ресурсов ИИ. Проверялись корректность команд, переходы между фазами, работа подсистем хранения данных, взаимодействие с Mini App и устойчивость при недоступности внешних сервисов LLM и TTS. По результатам испытаний критичные пользовательские сценарии выполняются без блокирующих дефектов; применение ИИ повышает интерактивность игрового процесса, при этом темп партии определяется характеристиками выбранной модели и вычислительной инфраструктуры.

Таким образом, цель выпускной квалификационной работы достигнута, поставленные задачи решены. Разработанный программный комплекс представляет собой основу для дальнейшего развития интеллектуальных игровых приложений, объединяющих классические механики «Мафии», чат-ориентированный интерфейс и возможности современных языковых моделей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. ГОСТ 19781-90. Обеспечение систем обработки информации программное. Термины и определения. – Москва: Издательство стандартов, 1990. – 14 с.
2. ISO 9241-11:2018. Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts. – Geneva: International Organization for Standardization, 2018. – 30 p.
3. Козлов Д. А. Игровой дизайн и механики / Козлов Д. А. – Санкт-Петербург: Питер, 2019. – 304 с.
4. Давыдов М. Л. Основы разработки компьютерных игр / Давыдов М. Л. – Санкт-Петербург: Питер, 2018. – 480 с.
5. Шелл Дж. Геймдизайн. Как создать игру, в которую будут играть все / Шелл Дж. – Москва: Альпина Паблишер, 2020. – 640 с.
6. Бейтс Б. Геймдизайн: создание компьютерных игр / Бейтс Б. – Москва: Эксмо, 2019. – 400 с.
7. Жуков С. А. Разработка стратегий в компьютерных играх / Жуков С. А. – Москва: Альфа-книга, 2020. – 256 с.
8. Яковлев М. В. Алгоритмы и структуры данных для игр / Яковлев М. В. – Москва: Питер, 2022. – 368 с.
9. Михайлов Д. Ю. Оптимизация игровых алгоритмов / Михайлов Д. Ю. – Санкт-Петербург: Питер, 2020. – 304 с.
10. Хромов П. А. Проектирование пользовательских интерфейсов в играх / Хромов П. А. – Москва: Интуит, 2021. – 224 с.
11. Орлов А. С. Разработка игровых сценариев и сюжетов / Орлов А. С. – Москва: Альфа-книга, 2021. – 288 с.
12. Романов И. В. Интерактивные технологии в играх / Романов И. В. – Москва: Питер, 2023. – 320 с.
13. Савельев О. Н. Архитектура программных систем / Савельев О. Н. – Москва: Питер, 2023. – 400 с.
14. Васильев И. Н. Программирование игр / Васильев И. Н. – Москва: ДМК Пресс, 2019. – 560 с.
15. Тихонов Е. С. Игровые движки: обзор и применение / Тихонов Е. С. – Санкт-Петербург: Питер, 2019. – 320 с.
16. Соколов А. П. Платформы для разработки игр / Соколов А. П. – Москва: ИНФРА-М, 2021. – 256 с.
17. Иванов В. П. Telegram-боты: разработка и применение / Иванов В. П. – Москва: Интуит, 2021. – 192 с.

18. Григорьев П. С. Создание игровых ботов / Григорьев П. С. – Москва: ИНФРА-М, 2022. – 288 с.
19. Смирнов И. А. Разработка сетевых игр / Смирнов И. А. – Москва: ДМК Пресс, 2020. – 480 с.
20. Таненбаум Э., ван Стеен М. Распределенные системы: принципы и парадигмы / Таненбаум Э., ван Стеен М. – Санкт-Петербург: Питер, 2021. – 896 с.
21. Telegram Bot API: [документация] // Telegram: [официальный сайт]. – [2026]. – URL: <https://core.telegram.org/bots/api> (дата обращения: 06.06.2026). – Режим доступа: свободный.
22. Getting updates: [документация] // Telegram: [официальный сайт]. – [2026]. – URL: <https://core.telegram.org/bots/api#getting-updates> (дата обращения: 06.06.2026). – Режим доступа: свободный.
23. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. – Санкт-Петербург: Питер, 2020. – 368 с.
24. Ушаков Д. А. Разработка мобильных игр / Ушаков Д. А. – Москва: Эксмо, 2023. – 304 с.
25. Digital 2024: Russia // DataReportal: [official website]. – [2024]. – URL: <https://datareportal.com/reports/digital-2024-russia> (дата обращения: 06.06.2026). – Режим доступа: свободный.
26. Российский рынок мессенджеров: аналитика и тренды // Brand Analytics: [официальный сайт]. – [2024]. – URL: <https://br-analytics.ru/blog/> (дата обращения: 06.06.2026). – Режим доступа: свободный.
27. Морозов Е. В. Разработка игр с использованием Python / Морозов Е. В. – Москва: ИНФРА-М, 2020. – 320 с.
28. Соколов В. П. Программирование на Python для игровых приложений / Соколов В. П. – Москва: ИНФРА-М, 2020. – 384 с.
29. Крылов Е. А. Разработка мультиплеерных игр / Крылов Е. А. – Москва: ИНФРА-М, 2022. – 352 с.
30. Чиксентмихайи М. Поток: психология оптимального переживания / Чиксентмихайи М. – Москва: Альпина нон-фикшн, 2022. – 464 с.
31. Лебедев Р. С. Искусственный интеллект: от основ к продвинутым техникам / Лебедев Р. С. – Москва: ДМК Пресс, 2022. – 608 с.
32. Шишкин А. А. Искусственный интеллект для анализа данных / Шишкин А. А. – Москва: ДМК Пресс, 2023. – 512 с.

33. Рассел С., Норвиг П. Искусственный интеллект: современный подход / Рассел С., Норвиг П. – Москва: Вильямс, 2021. – 1408 с.
34. Бобров А. В. Искусственный интеллект в играх / Бобров А. В. – Санкт-Петербург: БХВ-Петербург, 2020. – 416 с.
35. Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение / Гудфеллоу Я., Бенджио И., Курвилль А. – Москва: ДМК Пресс, 2018. – 652 с.
36. Егоров К. Д. Искусственный интеллект для разработчиков игр / Егоров К. Д. – Москва: Эксмо, 2023. – 352 с.
37. Ларин С. П. Искусственный интеллект и нейронные сети в играх / Ларин С. П. – Москва: Эксмо, 2021. – 448 с.
38. Петров А. И. Искусственный интеллект в стратегических играх / Петров А. И. – Санкт-Петербург: БХВ-Петербург, 2021. – 384 с.
39. Федоров В. В. Искусственный интеллект в компьютерных играх: современные подходы / Федоров В. В. – Москва: Альфа-книга, 2020. – 288 с.
40. Миллингтон Я., Фанге Дж. Искусственный интеллект в компьютерных играх / Миллингтон Я., Фанге Дж. – Москва: Вильямс, 2019. – 880 с.
41. Соммервилл И. Инженерия программного обеспечения / Соммервилл И. – Москва: Вильямс, 2020. – 928 с.
42. WerewolfBot: [Telegram-бот] // Telegram: [мессенджер]. – [2026]. – URL: <https://t.me/werewolfbot> (дата обращения: 06.06.2026). – Режим доступа: свободный.
43. TruthOrLieBot: [Telegram-бот] // Telegram: [мессенджер]. – [2026]. – URL: <https://t.me/truthorliebot> (дата обращения: 06.06.2026). – Режим доступа: свободный.
44. PlayQuestBot: [Telegram-бот] // Telegram: [мессенджер]. – [2026]. – URL: <https://t.me/playquestbot> (дата обращения: 06.06.2026). – Режим доступа: свободный.
45. Montfort N. Twisty Little Passages: An Approach to Interactive Fiction / Montfort N. – Cambridge: MIT Press, 2005. – 312 p.
46. Reigns: [мобильная игра] // Devolver Digital: [official website]. – Austin, [2016]. – URL: <https://www.devolverdigital.com/games/reigns> (дата обращения: 06.06.2026). – Режим доступа: свободный.
47. A Dark Room: [браузерная игра] // Doublespeak Games: [official website]. – [2013]. – URL: <https://adarkroom.doublespeakgames.com/> (дата обращения: 06.06.2026). – Режим доступа: свободный.